

Recommender Systems Homework Report — Assignment A01 (Random Lunch Generator)

Status: Report

Name: Abaidullah Seikh

Email: abaidullahseikh06@gmail.com

Date: 2026-09-16

Assignment: A01 — Random Lunch Generator (Week 1)

Abstract

The provided Random Lunch Generator sometimes displayed the selected lunch name without its food icon, even though the recommendation itself completed. I used OpenCode to inspect the original prompt and code, audit all 12 Font Awesome mappings against the exact Free 6.4.0 dependency, and reproduce the suspected cases deterministically in Chrome before and after a minimal fix. The audit found 3 unsupported mappings out of 12 (25% of the menu under the intended equal-probability baseline), and after changing only those three classes, all four controlled browser cases rendered correctly with valid `::before` content, HTTP 200 for the CSS and WOFF2 font, and no captured console or network errors. The main lesson for me was that plausible AI-generated dependency names still need exact-version and runtime verification before I accept them as correct.

1. Problem Definition & References (Introduction)

I started from the exact Week 1 starter application rather than building a different version. The official course Week 1 GitHub folder contains the supplied `prompt.md` and `index.html` [2]. The app contains 12 lunch options and selects one with `Math.floor(Math.random() * lunchMenu.length)`. After a 500 ms “Thinking” state, it writes the selected lunch name and a Font Awesome `<i>` element into the display. The course task was to investigate why icons sometimes do not display, find the cause in the provided artifacts, fix it, and prove the result.

The key observation was that the recommendation logic still worked: the lunch name appeared. The failure was visual. Because the item is selected randomly, an item-specific icon defect can look like an intermittent application problem. I therefore separated the selection logic from the rendering problem and asked whether the missing icon came from the dependency, resource loading, animation, timing, or the configured icon class.

```
[ Observed symptom ]
random lunch selected -> name appears -> some selections have no glyph
|
v
[ Investigation question ]
randomness? CDN/font? animation? timing? invalid icon mapping?
```

Personal hand-check: after the dependency audit identified 3 broken mappings out of 12 items, I calculated $3 / 12 = 0.25 = 25\%$. This is a simple theoretical exposure rate under the intended equal-probability baseline, not a measured click-frequency benchmark.

Research question: Which part of the supplied app causes the missing-icon symptom, can I reproduce it deterministically, and can I fix it with a minimal change that survives a before/after verification? Verified problem: the starter application contains three icon mappings that are unsupported in the exact Font Awesome Free 6.4.0 dependency used by the page; for those items, the browser produces no generated `::before` glyph. This conclusion is grounded in the official starter source [2], the exact stylesheet [3], the version-pinned Font Awesome metadata [4], and CSS generated-content behavior [5].

Evidence chain: I first established the baseline from the official starter code, then audited all 12 mappings against the pinned dependency, reproduced the three predicted failures with a supported Pizza control, changed only the three invalid mappings, rejected an inconclusive CDN-timeout run, and finally repeated the same browser verification successfully. This sequence separates problem identification from solution verification and makes both claims independently checkable.

2. Related Work & Deep Dive: Bugs and Technical Root Causes

2.1 References and alternatives

The starter prompt says that each recommendation should be paired with a relevant icon or image, but it does not pin an icon-library version or require every generated icon name to be validated [2]. The accompanying starter index.html loads Font Awesome Free 6.4.0 from cdnjs [2], so I treated the exact 6.4.0 stylesheet [3] and the official version-pinned Font Awesome 6.4.0 icon metadata [4] as the primary dependency references. For the generated-content mechanism itself, I used MDN's CSS content documentation as an independent browser/CSS reference [5].

Before changing anything, I considered three implementation alternatives: upgrade Font Awesome until the original names happened to work, replace the icon system with local images, or preserve the provided dependency and correct only unsupported mappings. I chose the third option because it changes one variable, keeps the starter architecture intact, and makes the fix easier to verify.

2.2 Exact audit of all 12 menu mappings

I instructed OpenCode to audit every configured class against the exact loaded dependency and not to rely on memory. This step corrected an early suspicion: Burger and Tacos use older-looking names, but the stylesheet proves that both are valid aliases. Only Ramen, Pasta, and Soup were unsupported in Font Awesome Free 6.4.0.

Table 1. Version-pinned audit of all 12 lunch icon mappings

Lunch	Configured class	Status	Evidence / role
Pizza	fa-pizza-slice	Supported	Canonical; U+F818
Sushi	fa-fish	Supported	Canonical; U+F578
Burger	fa-hamburger	Supported	Alias of burger; U+F805
Salad	fa-leaf	Supported	Canonical; U+F06C
Tacos	fa-utensil-spoon	Supported	Alias of spoon; U+F2E5
Ramen	fa-bowl-hot	Unsupported	No CSS selector / metadata entry
Sandwich	fa-bread-slice	Supported	Canonical; U+F7EC
Pasta	fa-pasta	Unsupported	No CSS selector / metadata entry
Curry	fa-mortar-pestle	Supported	Canonical; U+F5A7
Steak	fa-drumstick-bite	Supported	Canonical; U+F6D7
Soup	fa-bowl	Unsupported	No CSS selector / metadata entry
BBQ	fa-fire	Supported	Canonical; U+F06D

2.3 Technical root cause

Font Awesome applies the solid font through the fas class and generates the icon character through a matching ::before rule [3]. A supported selector such as fa-pizza-slice contributes a concrete content value (U+F818). The three unsupported selectors have no matching generated-content rule in the pinned dependency, so the <i> element can exist in the DOM while the browser still has no glyph to draw. This interpretation is consistent with CSS generated-content behavior: the content property determines what a pseudo-element renders, and a computed value of none produces no generated content [5].

[Confirmed failure path]		
unsupported class -> no matching ::before content -> empty glyph area		
Ramen: fa-bowl-hot	Pasta: fa-pasta	Soup: fa-bowl

2.4 Competing explanations I kept open

I did not treat the static audit as enough by itself. I also considered a failed CDN stylesheet, a failed Font Awesome font request, delayed font loading, the fade animation, rapid-click timeout interactions, and a mismatch between the supplied file and a hosted copy. These alternatives were carried into the browser experiment so that the final explanation would be based on observed runtime evidence rather than on a plausible-sounding code review.

Table 2. Competing explanations and evidence from the controlled pre-fix test

Alternative explanation	Expected evidence if true	What my controlled test showed
General CDN/CSS failure	Many or all Font Awesome icons fail; CSS request fails	Pizza rendered; CSS returned 200 in the successful pre-fix run
Solid font failure	Selectors exist but glyph font is unavailable	Solid font loaded; Pizza rendered with FA 6 Free / 900
Animation hides result	Opacity remains low or icon appears after animation	Display opacity reached 1; failing glyphs still absent
Invalid item mapping	Same lunch fails repeatedly while control works	Ramen/Pasta/Soup failed consistently; Pizza worked

3. Method & Systematic Verification

3.1 How I managed the AI investigation

I deliberately directed OpenCode in stages. First I told it to read prompt.md and index.html without modifying files and to give me an investigation plan. Next I asked for an exact 12-item dependency audit. Then I required deterministic browser reproduction and explicitly told the agent not to call the root cause confirmed until the browser evidence matched the audit. Only after that did I allow a minimal source edit. This sequence made the workflow reproducible and kept the reasoning visible instead of jumping directly to a final fix.

3.2 Deterministic browser experiment

Instead of clicking randomly until the desired lunches happened to appear, I directed OpenCode to force four exact selections in headless Google Chrome 153.0.8010.36. Math.random() was overridden only inside the browser before the application ran; index.html was not edited for this test.

Math.random = () => (index + 0.5) / 12;					
Pizza = 0	Ramen = 5	Pasta = 7	Soup = 10		

Pizza was the supported control. Ramen, Pasta, and Soup were the three mappings predicted to fail. Each case was checked three times: once on page load and twice after pressing Generate. The test waited for document.fonts.ready, the application's 500 ms delay, and animation completion. It recorded the generated <i> class, visible rendering, computed ::before content, font family, font weight, element size, opacity, CSS/font network responses, and browser errors.

3.3 Before-fix results

Table 3. Deterministic browser results before the fix

Lunch	Class	Visible?	::before	Font / weight	CSS / WOFF2
Pizza	fa-pizza-slice	Yes	U+F818	Font Awesome 6 Free / 900	200 / 200
Ramen	fa-bowl-hot	No	none	Font Awesome 6 Free / 900	200 / 200
Pasta	fa-pasta	No	none	Font Awesome 6 Free / 900	200 / 200
Soup	fa-bowl	No	none	Font Awesome 6 Free / 900	200 / 200

All three measurements for each item agreed. Pizza rendered a visible glyph. Ramen, Pasta, and Soup consistently produced no glyph and ::before = none, while the lunch names still appeared and the lunch display opacity reached 1. The Font Awesome CSS and solid WOFF2 font both loaded successfully, and there were no relevant console or network errors. This matched the dependency audit and confirmed the code-level root cause in my controlled test.

3.4 Minimal fix

After the root cause was confirmed, I approved only the smallest justified source change. The layout, animation, 12-item menu, random-selection logic, and Font Awesome version remained unchanged.

Table 4. Minimal icon-mapping changes applied after root-cause confirmation

Lunch	Before	After	Verified FA 6.4.0 content
Ramen	fa-bowl-hot	fa-bowl-rice	U+E2EB
Pasta	fa-pasta	fa-utensils	U+F2E7
Soup	fa-bowl	fa-spoon	U+F2E5

Why this design matters: changing only three icon strings gave me a clean controlled intervention. If the same failing cases changed from no glyph to visible glyphs while Pizza stayed stable, the before/after evidence would be easy to interpret.

3.5 Controlled variables and evidence recorded

Table 5. Controlled variables and recorded evidence

Element	How I controlled or recorded it
Selection	Forced exact indices 0, 5, 7, and 10 instead of relying on random clicks
Control case	Pizza, a known-supported selector, was tested alongside the three failures
Dependency	Same Font Awesome Free 6.4.0 URL before and after
Timing	Waited for document.fonts.ready, the 500 ms app delay, and animation completion
Repetition	Page load plus two Generate clicks for each selected lunch
Evidence	DOM class, ::before content, font family/weight, visibility/size, HTTP responses, errors

4. Solution & Verifiable Evidence: Before vs. After

4.1 The first post-fix attempt did not count as success

The first browser rerun after the code change was blocked by CDN timeouts. The log showed `net::ERR_TIMED_OUT`, and even the unchanged Pizza control was affected by resource loading. I did not report this run as proof that the fix worked or failed. I preserved it as a separate failed attempt and asked OpenCode to repeat the exact verification without modifying `index.html` again.

What this taught me: a failed browser test is not automatically evidence against the newest code change. The control case helped separate an external resource failure from a code regression.

4.2 Successful verification retry

For the retry, I required the Font Awesome CSS and `fa-solid-900.woff2` downloads to complete with HTTP 200, `document.fonts.ready` to resolve, the application delay and animation to finish, the selected icon to be visible, and no relevant console/network error to be captured. All four cases met those criteria. Verified solution: only the three unsupported mappings were replaced, and the same controlled test changed Ramen, Pasta, and Soup from `::before = none` / no glyph to valid generated content and visible icons, while the unchanged Pizza control continued to render correctly.

Table 6. Successful post-fix browser verification

Lunch	Before	After	Post-fix ::before	CSS	WOFF2	Errors
Pizza	Visible	Visible	U+F818	200	200	None
Ramen	No glyph	Visible	U+E2EB	200	200	None
Pasta	No glyph	Visible	U+F2E7	200	200	None
Soup	No glyph	Visible	U+F2E5	200	200	None

BEFORE	
Ramen: <code>fa-bowl-hot</code>	-> <code>content:none</code> -> no glyph
Pasta: <code>fa-pasta</code>	-> <code>content:none</code> -> no glyph
Soup: <code>fa-bowl</code>	-> <code>content:none</code> -> no glyph
AFTER	
Ramen: <code>fa-bowl-rice</code>	-> <code>U+E2EB</code> -> visible
Pasta: <code>fa-utensils</code>	-> <code>U+F2E7</code> -> visible
Soup: <code>fa-spoon</code>	-> <code>U+F2E5</code> -> visible

4.3 Repetition and source-integrity checks

Each of the four post-fix cases again had three agreeing measurements, giving 12 successful post-fix observations across the controlled set. I do not generalize this to a visual test of all 12 menu items: all 12 were audited against the dependency, while the browser experiment intentionally focused on one supported control and the three confirmed failures.

The successful retry also checked source integrity. The fixed `index.html` had the same SHA-256 checksum before and after the retry, so the pass did not depend on another hidden edit. OpenCode also preserved screenshots, measurement logs, raw browser-event records, a source-integrity record, and the reproduction script in separate before, timeout, and retry evidence directories.

Table 7. Preserved verification artifacts and their purpose

Evidence artifact	Purpose
Screenshots	Visual confirmation of the selected lunch and icon state
Measurement log	Computed-style and HTTP measurements for the controlled cases
Browser-event log	Low-level network and browser evidence for each controlled case
Source-integrity record	Checksum evidence that verification did not change the source
<code>lunch_browser_audit.py</code>	Reproduction procedure used to force selections and collect measurements

4.4 Prompt improvement based on the observed failure

The official starter prompt asks for a relevant icon or image but does not require the coding model to validate icon names against the exact dependency version [2]. I therefore propose the following guardrails: pin Font Awesome Free 6.4.0; verify every class against the exact stylesheet and metadata [3], [4]; do not invent plausible icon names; use a verified fallback if a food-specific icon is unavailable; and visually test every lunch item before considering the page complete.

I keep the causal claim narrow: the code-level root cause is experimentally verified. The prompt weakness is a plausible prevention explanation, but I did not run repeated original-prompt versus revised-prompt generations, so I do not claim that the prompt directly caused those exact three invalid names.

5. Discussion, Improvement, AI Usage, and Conclusion

5.1 Failure, root cause, fix, and what worked

The original failure was deterministic at the item level even though it looked intermittent to a user. The three failing lunches referenced icon classes that did not exist in the exact Font Awesome Free 6.4.0 distribution. The strongest part of the workflow was the agreement between independent evidence layers: exact dependency audit, browser `::before` inspection, a stable Pizza control, a minimal three-line source change, and the repeated post-fix browser test.

The temporary CDN timeout was also valuable because it exposed a different failure mode. If I had treated every post-edit failure as a regression, I would have reached the wrong conclusion. Preserving that failed attempt and retrying without another source change improved the credibility of the final result.

5.2 Limits and next recommender improvement

This homework verifies the missing-icon defect and its fix. It does not evaluate the statistical quality of JavaScript `Math.random()` or user satisfaction with the recommendations. The 25% calculation is only a theoretical exposure rate for the three broken mappings under the intended equal-probability baseline.

A next RecSys-oriented experiment I would test is preventing the same lunch from appearing twice in a row, or treating an immediate second Generate click as weak negative feedback and temporarily reducing the previous item's probability. I did not implement or benchmark this idea in the current homework, so I report it only as future work rather than as a completed improvement.

5.3 AI usage and my personal verification

I used OpenCode 1.18.31 with the build agent and OpenAI gpt-6-astra (medium). OpenCode is an open-source AI coding agent [6]. I used it for source inspection, dependency auditing, deterministic browser testing, the minimal three-class patch, and the verification retry. I did not ask OpenCode to solve everything in one step; I controlled the sequence of investigation, testing, editing, and retesting.

I managed the AI step by step instead of asking it to fix the application immediately: understand first, audit the dependency, reproduce the failure, confirm the mechanism, apply the smallest change, and retest. I also required the agent not to call the post-fix run successful when the CDN timed out.

5.4 What worked, what surprised me, and what remains limited

What worked: using a supported control, forcing exact indices, and checking `::before` made the symptom reproducible instead of anecdotal. The minimal three-string patch then made the before/after comparison easy to interpret.

What surprised me: the invalid names were plausible enough to look correct during casual review, and the first post-fix run failed for a network reason unrelated to the code change. Both cases showed me why I should verify mechanisms instead of trusting surface plausibility.

Limitations: the browser experiment focused on four items rather than visually retesting all 12 after the fix, and I did not run a controlled prompt-generation benchmark. I therefore keep claims about all-menu visual behavior and prompt causality narrower than the evidence supports.

For my personal verification, I manually checked $3 / 12 = 0.25 = 25\%$ and compared the three changed icon mappings in the before/after diff. I also checked that the report tables are consistent with the OpenCode results. The detailed Chrome measurements were produced by the OpenCode-driven browser test, while my role was to define the test, compare the evidence, and decide when the result was sufficiently verified.

5.5 Conclusion

My final conclusion is that the missing-icon symptom came from three unsupported Font Awesome Free 6.4.0 mappings, not from the random lunch-selection logic. The problem is supported by authoritative references to the official starter artifacts and the exact dependency, and the solution is supported by reproducible before/after browser evidence. I reached this conclusion through a complete chain: version-pinned dependency audit, deterministic pre-fix reproduction, minimal three-class fix, an honestly rejected timeout run, and a successful controlled retry. For me, the main learning outcome was methodological: with AI-assisted coding, a plausible explanation remains a hypothesis until the exact dependency, runtime behavior, and repeated evidence agree.

References

- [1] S. Jin, "LLM-4-RECSYS," Week 1 lecture slides, HSE University, 2026.
- [2] S. Jin, "RecSys-LLMs - Week 1," GitHub repository, 2026. [Online]. Available: <https://github.com/dryjins/RecSys-LLMs/tree/main/week1>. Accessed: Sep. 16, 2026.
- [3] cdnjs, "Font Awesome 6.4.0 - all.min.css," 2026. [Online]. Available: <https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css>. Accessed: Sep. 16, 2026.
- [4] Font Awesome, "icons.json - Font Awesome 6.4.0 metadata," FortAwesome/Font-Awesome. [Online]. Available: <https://github.com/FortAwesome/Font-Awesome/blob/6.4.0/metadata/icons.json>. Accessed: Sep. 16, 2026.
- [5] MDN Web Docs, "content - CSS," Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/content>. Accessed: Sep. 16, 2026.
- [6] OpenCode, "OpenCode Documentation," 2026. [Online]. Available: <https://opencode.ai/en/docs>. Accessed: Sep. 16, 2026.